Documentación — Compuclinica

Compuclinica

1. Identidad y alcance

- **Nombre:** Compuclinica.
- **Descripción:** Aplicación de escritorio para gestionar compras, inventario de repuestos, proveedores, técnicos, órdenes de servicio y ventas a clientes.
- **Tipo:** Aplicación Java Swing local.
- **Versión:** 1.0-SNAPSHOT.
- **Autor:** Julián Forero.
- **Repositorio:** Privado.
- **Usuarios objetivo:** Personal administrativo o técnico de un taller de reparación de computadores.
- **Base de datos:** No utiliza una base de datos relacional. Persiste información en archivos binarios `.dat`.

2. Stack tecnológico

- Java 21.
- Maven.
- Swing/AWT para la interfaz gráfica.
- NetBeans GUI Builder.
- Serialización Java mediante `ObjectInputStream` y `ObjectOutputStream`.
- Configuración principal: `pom.xml`, `nbactions.xml` y `ui_config.properties`.

3. Arquitectura

Estructura modular por dominio funcional (proveedores, técnicos, repuestos, compras, órdenes de servicio, ventas), cada uno con su propia vista, controlador y DAO.

4. Entrada y ejecución

La entrada funcional principal está en `Gestor.java`.

El menú principal abre:

- Compras.
- Órdenes de servicio.
- Ventas a clientes.
- Repuestos/inventario.
- Proveedores.
- Técnicos.

Cada controlador crea su DAO, registra listeners, carga datos y muestra su vista. Las vistas también contienen métodos `main` independientes para abrirlas directamente.

5. Módulos funcionales

Proveedores

CRUD básico de proveedores: crear, consultar, actualizar y eliminar. Identificador calculado como máximo existente más uno. Modelo: id, nombre.

Técnicos

CRUD básico de técnicos: crear, consultar, actualizar y eliminar. Modelo: id, nombre.

Repuestos

Gestión del inventario: nombre, serial, precio, tipo (Garantía o Propio), stock, cálculo de precio total, aumento y descuento de inventario. Prevención parcial del borrado cuando el repuesto está referenciado.

Compras

Permite registrar proveedor, fecha, repuesto y cantidad. Al registrar una compra se incrementa el stock del repuesto. Archivo persistente: `compra.dat`.

Órdenes de servicio

Permite registrar cliente, técnico, hasta cuatro repuestos, número de orden y fecha. Al crear una orden se descuenta stock de los repuestos utilizados. Archivo persistente: `ordenservicio.dat`.

Ventas a clientes

Permite registrar cliente, fecha, hasta cuatro repuestos, cantidad por repuesto y total calculado. Al registrar una venta se descuenta el inventario. Archivo persistente: `ventacliente.dat`.

6. Persistencia

Cada DAO mantiene una lista de objetos y sobrescribe el archivo completo después de cada operación.

Características:

- Rutas relativas al directorio de ejecución.
- Serialización nativa de Java.
- Las entidades implementan `Serializable`.

7. Flujo de datos de ejemplo

- **Registrar una compra:** se guarda el registro de compra y se incrementa el stock del repuesto correspondiente.
- **Registrar una venta:** se guarda el registro de venta y se descuenta el stock de los repuestos vendidos.
- **Registrar una orden:** se guarda la orden de servicio y se descuenta el stock de los repuestos utilizados.

8. Interfaz y tematización

Las vistas son `JFrame` generadas con NetBeans.

Componentes principales: `JTable`, `JTextField`, `JComboBox`, `JButton`, selectores de fecha, diálogos `JOptionPane`.

`ThemeManager.java` aplica colores y fuentes recursivamente.

9. Instalación y ejecución

Requisitos:

- JDK 21.
- Maven instalado.
- Sistema con entorno gráfico.
- Permisos de lectura/escritura en el directorio de ejecución.

10. APIs e integraciones

No existen: API REST, GraphQL, Webhooks, WebSockets, autenticación, integraciones externas, servicios de terceros, analítica, sistema de notificaciones, ni CDN o hosting web.

11. Seguridad y privacidad

No hay mecanismos de login, roles o permisos, JWT/OAuth, cifrado, HTTPS, CSRF, rate limiting, validación centralizada, gestión segura de secretos, auditoría de cambios, política de privacidad, ni retención o eliminación de datos.

Los archivos `.dat` pueden contener información local sin protección y son susceptibles a corrupción o manipulación.

12. Testing y calidad

La validación actual depende de controles implementados en los controladores y pruebas manuales mediante la interfaz.

13. Riesgos conocidos

- Persistencia no atómica: una operación puede actualizar stock y fallar al guardar la operación relacionada.
- IDs calculados con máximo + 1, inseguros ante concurrencia.
- Posibilidad de cantidades, precios o stocks inválidos.
- Fechas almacenadas como texto.
- Una orden puede repetir el mismo repuesto y producir descuentos incorrectos.
- Proveedores y técnicos pueden eliminarse aunque estén relacionados con registros existentes.
- Los objetos relacionados se guardan como snapshots serializados, no como referencias normalizadas.
- Ruta absoluta del logo, lo que reduce la portabilidad.
- Falta de `serialVersionUID`.
- La ventana principal se crea fuera de `EventQueue.invokeLater`.
- No hay estrategia de backup o recuperación.
- El error de escritura se imprime en consola y no siempre se informa al usuario.

14. Troubleshooting

Maven no encontrado: instalar Maven y verificar la instalación.

Clase principal no encontrada: usar la clase real de entrada, no el valor declarado actualmente en `pom.xml`.

Datos no aparecen: verificar que los archivos `.dat` estén en el directorio actual desde el que se ejecuta Java.

Datos perdidos o corruptos: los archivos binarios se sobrescriben completamente; restaurar una copia previa si existe. No hay recuperación integrada.

15. Deployment

El proyecto está preparado como aplicación JAR local, no como aplicación web.

No hay staging, producción remota, Docker, Kubernetes, GitHub Actions, releases, instalador, actualizaciones automáticas, monitoreo, uptime checks, backup remoto ni rollback automatizado.